

Chapter 17

애플리케이션 개발을 위한 모범 사례

Best practices for developing apps

발표자: 개발자 이프

2026



오늘 다룰 내용

1. 17장의 위치 + 1판 ↔ 2판 매핑
 2. 종합 그림 — 한 앱이 살아가는 데 필요한 것들 (17.1)
 3. 파드 라이프사이클 — 시작/종료 흐름 (17.2)
 4. 📡 **Demo 1:** Graceful shutdown 효과 비교
 5. 안전한 시작/종료 — Probe + preStop 협력 (17.3)
 6. 이미지/라벨/로그/annotation 모범사례 (17.4)
 7. 개발/CI/CD (17.5)
 8. 📡 **Part 7:** atlas-backend 매니페스트 실 사례 분석
 9. 종합
- 🕒 약 45분 + Q&A

이전 발표와의 연결

```
[3장]  Pods      → 파드 단위, Labels, Namespaces
[13장] Security   → SecurityContext, PSA, NetworkPolicy
[16장] StatefulSets → 순서 있는 stateful 워크로드
[17장] Best Practices → ← 오늘
      ↓
      "지금까지 배운 거 종합 + 모범 사례 + 우리 클러스터 실 사례"
```

- 17장 = 책의 종합 챕터
- 새 토픽보다 이미 배운 토픽들을 모범 사례 관점에서 다시

17장의 위치 — 왜 책의 마지막에?

1~16장: 각각의 K8s 객체 (Pod, Service, Deployment, ConfigMap, ...)

17장: 그것들을 모아 진짜 앱 만들 때 알아야 할 것들

답해야 할 질문들:

- 파드는 언제든 죽을 수 있다는 가정에서 앱을 어떻게 짜야 하나?
- 트래픽 받는 도중 파드가 죽으면 사용자 요청은?
- 이미지 태그 / 라벨 / 로그는 어떻게?
- 개발 시 매니페스트는 어떻게 관리?

1판 17장 ↔ 2판 매핑

2판에는 1판 17장에 해당하는 단일 챕터가 없습니다. 토픽이 여러 장에 분산.

1판 17장 절	2판 위치
17.1 Bringing it all together (종합 그림)	(없음) — 본문 전체에 녹아있음
17.2 파드 라이프사이클	2판 6장 Managing the Pod lifecycle
17.3 클라이언트 요청 처리 (probe + 종료)	2판 6장 + Services 챕터
17.4.1~2 이미지 모범사례	2판 4장 Pod 객체 + 7장 컨테이너
17.4.3 multi-dimensional labels	2판 10장 Namespaces & Labels
17.4.4 Annotations	2판 10장
17.4.5 process termination 정보	2판 6장
17.4.6 logs	2판 부분적 (운영 영역)
17.5 개발/CI/CD	2판에 약함 (운영 영역)

💡 2판은 운영(13~16장)을 별도 영역으로 분리. 17장 "모범사례" 같은 종합 챕터를 없앴.

Part 2

Bringing it all together

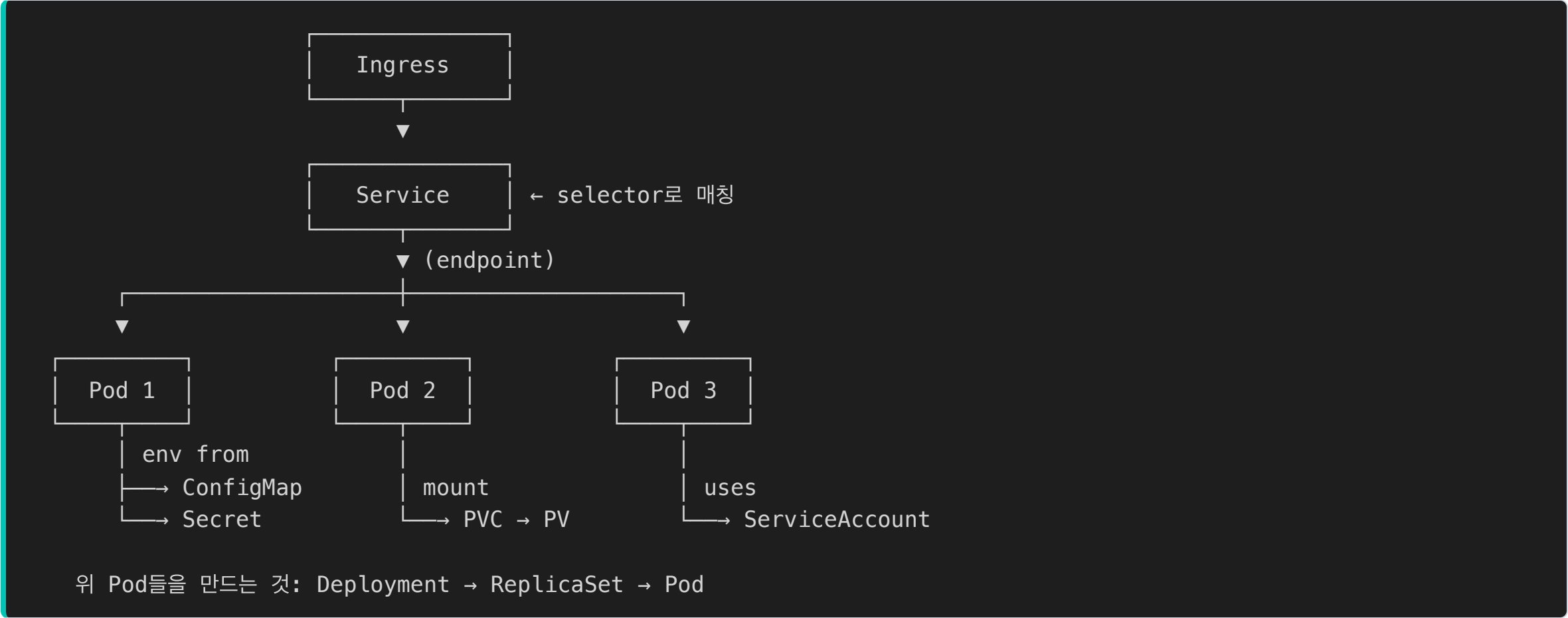
17.1 — 한 앱이 살아가는 데 필요한 것들

한 앱을 살리는 데 필요한 객체 8종

책임	객체
실행 단위	Pod
복제·자동 재생성	Deployment (→ ReplicaSet → Pod)
안정적 진입점	Service
외부 노출	Ingress
환경 변수/설정	ConfigMap
비밀 값	Secret
영구 저장소	PVC + PV (or StorageClass)
신원/권한	ServiceAccount + RBAC

→ "한 앱 만들기 = 위 8종을 적절히 묶는 일".

관계 다이어그램



"왜 객체가 이렇게 많지?"

각 객체는 하나의 관심사만 책임:

- **Pod** = 실행 단위 (volatile)
- **Deployment** = "어떤 파드를 몇 개" (declarative state)
- **Service** = "이름 → 파드들" (stable endpoint)
- **ConfigMap/Secret** = "환경별 설정 vs 비밀"
- **PVC** = "디스크 요청"
- **SA** = "이 파드의 신원"

→ **Single Responsibility Principle**. 합치면 더 복잡해짐.

Part 3

파드 라이프사이클

17.2

핵심 전제: "앱은 언제든지 죽는다"

K8s에서 파드는 언제든지 강제 종료될 수 있음:

- 노드 장애
- 자원 부족 (OOMKilled, eviction)
- 노드 drain (정비, autoscaling)
- 사용자가 직접 delete
- Deployment rollout
- preemption (priority 낮은 거 밀려남)

→ 앱 설계 첫 가정: "내 파드는 곧 죽는다".

죽을 때 일어나는 일

1. delete 신호 (API Server)
 - DeletionTimestamp 설정
 - 파드 STATUS → Terminating
2. endpoint controller: 모든 Service에서 이 파드 빼냄
 - kube-proxy에 전파 (시간 걸림 – 핵심 포인트)
3. kubelet: preStop hook 실행 (있으면)
4. kubelet: SIGTERM → 컨테이너 메인 프로세스
5. terminationGracePeriodSeconds 동안 대기 (기본 30초)
6. 시간 지나면 SIGKILL → 강제 종료
7. 파드 객체 삭제

⚠ 2번과 4번이 동시에 일어남. endpoint 빠지기 전에 트래픽 받으면 문제.

postStart hook — 시작 직후 후크

```
lifecycle:
  postStart:
    exec:
      command: ["/bin/sh", "-c", "echo 'started' > /tmp/started"]
```

- 컨테이너 시작 직후 실행
- 실행 완료 전까지 컨테이너는 **Waiting** 상태
- 실패하면 컨테이너 재시작

⚠ 거의 안 씀. 시작 로직은 컨테이너 entrypoint나 init container가 더 명확.

preStop hook — 종료 직전 후크 (중요!)

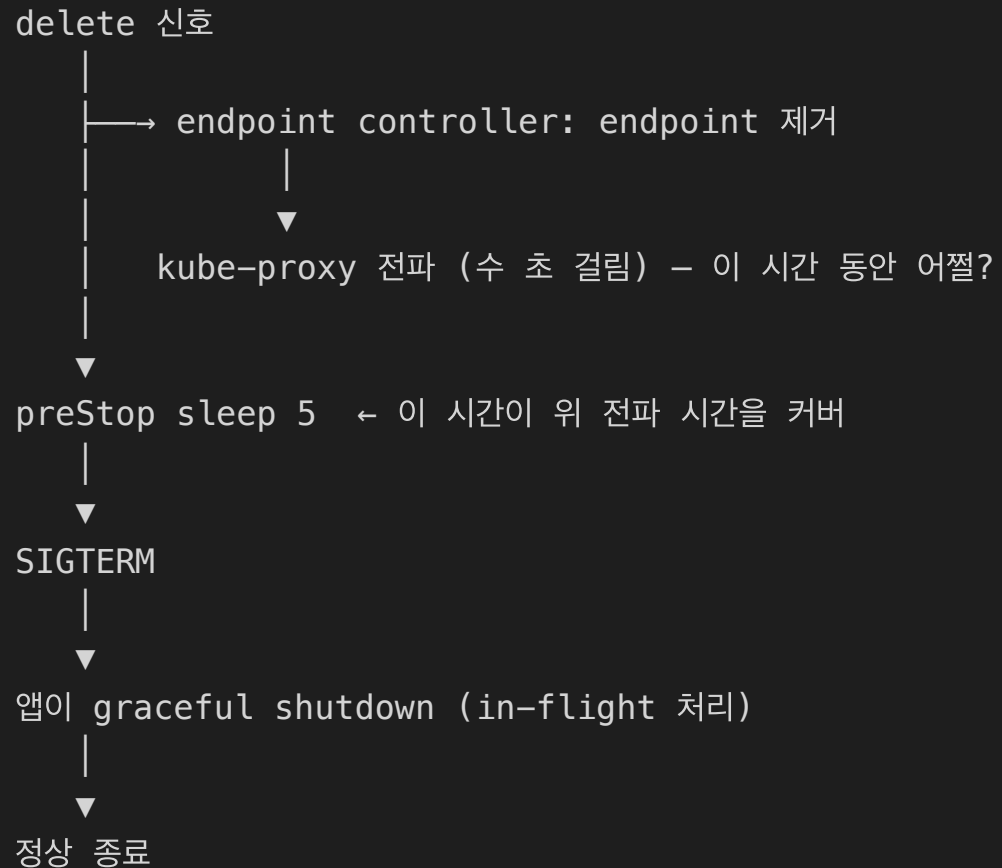
```
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 5"]
```

- SIGTERM 전에 실행
- preStop 완료될 때까지 SIGTERM 안 옴
- preStop도 terminationGracePeriodSeconds 안에서 카운트

용도:

1. **endpoint** 전파 시간 벌기 (가장 흔함, sleep 5~10초)
2. 정상 종료 신호 보내기 (Spring `actuator/shutdown` 호출 등)
3. 외부 리소스 정리 (락 해제, 등)

graceful shutdown 흐름 (좋은 패턴)



→ 새 요청은 다른 파드로, 진행 중 요청은 처리. 0 에러.

terminationGracePeriodSeconds

```
spec:
  terminationGracePeriodSeconds: 60    # 기본 30
```

- SIGTERM 후 SIGKILL까지의 시간
- 이 안에 preStop 실행 + 앱 정리 완료해야 함
- preStop이 30초 걸리면 grace = 60일 때 SIGTERM 후 30초 더 있음

💡 앱 종료 로직 길면 늘려야. DB connection 닫기, in-flight 처리 등.

길이 설계:

- 일반 웹: 30초 충분
- 메시지 큐 consumer: 60~120초 (진행 중 메시지 처리)
- 대형 stateful: 더 길게

시작 순서 보장은 K8s가 안 해줌

"DB 먼저, 그 다음 API" 같은 의존성:

❌ K8s는 보장 안 함 — Deployment끼리 순서 없음

- StatefulSet은 같은 set 안에서만 순서 (16장)

✅ 해결 방법:

1. **init container** — DB 떠있는지 wait

```
initContainers:  
- name: wait-for-db  
  image: busybox  
  command: ["sh", "-c", "until nc -z db 5432; do sleep 1; done"]
```

2. 앱 안에 재시도 로직 — DB 못 찾으면 백오프 후 재시도

3. **Readiness probe** — DB 연결 확인 후 ready



Demo 1

Graceful Shutdown 효과 비교

preStop 없는 vs 있는

Demo 1 셋업

```
# 1. Service + load-generator
kubectl apply -f demos/ch17-best-practices/03-service-and-load.yaml

# 2. 두 deployment 동시에 (총 4 pods)
kubectl apply -f demos/ch17-best-practices/01-app-bad.yaml
kubectl apply -f demos/ch17-best-practices/02-app-good.yaml

# 3. load-gen 로그 stream
kubectl logs -f load-gen -n k8s-demo
# 출력: 10:00:01 OK
#       10:00:02 OK
#       10:00:03 OK ...
```

→ 4개 파드 모두 정상 응답.

Demo 1: bad rollout — FAIL 발생

```
# 다른 터미널에서  
kubectl rollout restart deploy app-bad -n k8s-demo
```

load-gen 로그:

```
10:00:10 OK  
10:00:11 OK  
10:00:12 FAIL    ← 옛 파드 SIGTERM, 트래픽 일부 흘러감  
10:00:13 FAIL  
10:00:14 OK  
10:00:15 OK
```

→ preStop 없음 → endpoint 빠지기 전에 SIGTERM → **broken connection**.

Demo 1: good rollout — 무중단

```
kubectl rollout restart deploy app-good -n k8s-demo
```

load-gen 로그:

```
10:01:00 OK
10:01:01 OK
10:01:02 OK    ← preStop 동안 endpoint 미리 빠짐
10:01:03 OK    ← 트래픽이 다른 파드로 라우팅
10:01:04 OK
...
```

→ preStop sleep 5초가 endpoint 전파 시간을 커버 → **0 FAIL**.

```
# 정리
kubectl delete -f demos/ch17-best-practices/
```

Demo 1 요점

구분	preStop 없음 (bad)	preStop sleep 5 (good)
SIGTERM 시점	즉시	5초 후
endpoint 갱신	늦음 (SIGTERM 이후도)	preStop 동안 완료
in-flight 요청	일부 끊김	없음
5xx/timeout	발생	0

→ 5초의 사전 대기로 무중단 배포.

Part 4

안전한 시작/종료

17.3

시작 시 broken connection — Readiness probe로 방지

문제: 컨테이너 떴지만 아직 준비 안 됨 (DB 연결 중, 캐시 워밍, ...)

→ 그 사이 Service가 트래픽 보내면 5xx.

해결: **Readiness probe**

```
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

- Ready=true 될 때까지 **endpoint**에서 제외
- Ready=false 다시 가면 **endpoint**에서 빠짐

💡 시작/종료 모두에서 작동. 17장의 핵심 도구.

Liveness vs Readiness vs Startup

Probe	실패 시 동작	용도
Liveness	컨테이너 재시작	죽었거나 stuck됐을 때 회복
Readiness	endpoint에서 제외	일시적 not-ready 처리
Startup (1.16+)	시작 단계 완료까지 다른 probe 보류	느린 시작 앱 보호

세 개 함께 쓰는 패턴:

- **Startup:** failureThreshold 30 (= 30 × period 동안 대기)
- **Liveness:** 그 후 빈도 낮게 (period 30s)
- **Readiness:** 자주 (period 5s)

Probe 함정 ① — 같은 endpoint 쓰면?

자주 보는 패턴:

```
livenessProbe: { httpGet: { path: /health, ... } }  
readinessProbe: { httpGet: { path: /health, ... } }
```

→ liveness가 "DB 연결 안 됨" 같은 일시적 문제로 fail되면 컨테이너 재시작.

권장:

- **livenessProbe**: 정말 죽었는지 (e.g. `/health/live` — 프로세스만)
- **readinessProbe**: 트래픽 받을 수 있는지 (e.g. `/health/ready` — DB/cache 포함)

💡 endpoint 분리가 모범 — 그러나 단순한 앱은 같이 써도 동작.

Probe 함정 ② — 시간 설정

```
livenessProbe:
  initialDelaySeconds: 120    # 첫 체크까지
  periodSeconds: 30          # 체크 간격
  timeoutSeconds: 1           # 응답 대기
  failureThreshold: 3         # 연속 실패 횟수
```

= 최악의 경우 $120 + 30 \times 3 = 210$ 초 후에야 재시작.

너무 짧으면: 일시적 GC pause → 죽지 않은 컨테이너 재시작 (false positive)

너무 길면: 죽은 컨테이너 한참 살아있음 (재시작 지연)

💡 앱의 특성에 맞춰 조정. JVM은 GC pause 대비 longer timeout.

종료 시 broken connection — preStop으로 해결

다시 보는 흐름:

```
delete 신호
|
├── endpoint 빠짐 (kube-proxy 전파 시간)
|
▼
preStop sleep 5 ← endpoint 전파 시간 커버
|
▼
SIGTERM → 앱 graceful shutdown
|
▼
정상 종료 (0 에러)
```

핵심: preStop이 endpoint 전파 시간을 흡수.

→ Demo 1에서 시연한 그것.

Part 5

이미지 · 라벨 · 로그 · annotation

17.4

17.4.1 — 작은 이미지

```
# Bad (1.5GB)
FROM ubuntu:22.04
RUN apt-get install -y openjdk-17 ...

# Good (200MB)
FROM eclipse-temurin:17-jre-alpine
```

권장 base image:

- **alpine** — 일반 워크로드 (musl libc 호환 확인)
- **distroless** — Google, 보안 강화 (shell 없음)
- **scratch** — Go 정적 바이너리

작은 이미지의 이점:

- 빠른 풀 → 빠른 시작
- 적은 공격 표면 → 보안 
- 적은 디스크 사용

17.4.2 — 이미지 태그 (절대 latest ❌)

```
# ❌ Bad
image: myapp:latest

# ● OK
image: myapp:1.2.3

# ✅ Best
image: myapp:1.2.3-2026.06.13-a1b2c3d
```

latest 의 문제:

- 어떤 버전인지 모름 (rollback 불가)
- 매번 새 풀 (imagePullPolicy 자동 Always)
- 같은 태그가 다른 이미지 가리킬 수 있음

imagePullPolicy:

- **IfNotPresent** — 명시 태그용 (권장)
- **Always** — **:latest** 시 자동 적용
- **Never** — 로컬 빌드된 이미지만

17.4.3 — Multi-dimensional Labels

❌ 단일 라벨

```
labels:  
  app: atlas-backend
```

✅ 다차원 라벨 (책 권장)

```
labels:  
  app.kubernetes.io/name: atlas-backend  
  app.kubernetes.io/instance: atlas-prod  
  app.kubernetes.io/version: "1.2.3"  
  app.kubernetes.io/component: backend  
  app.kubernetes.io/part-of: atlas  
  app.kubernetes.io/managed-by: kubectl
```

왜 다차원?

- 환경별, 컴포넌트별, 버전별 필터링
- Helm/ArgoCD/Lens 호환
- 그룹별 dashboard 자동 생성

💡 처음부터 다차원으로. 나중에 추가 어려움.

17.4.4 — Annotations 활용

```
metadata:
  annotations:
    # 빌드 정보
    build.git-commit: "a1b2c3d"
    build.timestamp: "2026-06-13T10:00:00Z"
    build.ci-url: "https://ci.example.com/build/1234"

    # 운영 정보
    contact: "team-platform@example.com"
    description: "Atlas backend API server"

    # 컨트롤러 동작 (실용)
    prometheus.io/scrape: "true"
    nginx.ingress.kubernetes.io/rewrite-target: "/"
```

빌드 정보 annotation의 가치:

- 사고 시 "어떤 commit이 났는지" 즉시 확인
- `kubectl describe` 로 메타데이터 확인 가능
- 로그 grep할 때 commit과 매칭

17.4.5 — Process Termination 정보

K8s가 컨테이너 종료 이유 추적:

```
kubectl get pod X -o jsonpath='{.status.containerStatuses[0].lastState}'
# {"terminated":{"exitCode":137,"reason":"OOMKilled",...}}
```

exitCode	의미
0	정상 종료
1~127	앱 에러
128 + N	시그널 N으로 종료
137	SIGKILL (128+9) → OOMKilled 가능성
143	SIGTERM (128+15) → 정상 graceful

terminationMessagePath (기본 `/dev/termination-log`):

- 앱이 죽기 전 마지막 메시지 기록
- K8s가 reading해서 status에 반영

17.4.6 — Logs는 stdout/stderr로

K8s 권장:

```
앱 → stdout/stderr → 컨테이너 런타임 → 노드 로그 파일  
→ Fluent Bit 등이 수집 → S3/Elasticsearch
```

✗ 안 좋은 패턴:

- 파일에 로그 (`/var/log/app.log`)
- sidecar 띄워서 파일 → stdout 변환 (불필요)

✓ 좋은 패턴:

- 앱이 stdout/stderr에 JSON 형식 로그
- 컨테이너 런타임이 알아서 처리
- 수집기는 노드 레벨에서 동작

💡 Spring Boot, Node.js, Go 등 다 stdout 기본. 그대로 두기.

Part 6

개발 / CI / CD

17.5

17.5.1~2 — 로컬 개발 환경

K8s 없이 개발:

- 직접 실행 (DB는 docker run)
- IDE에서 디버그 가능

K8s 위에서 개발:

- **minikube** — 모든 K8s 기능, 무거움
- **kind** — Docker 안에 K8s, 가벼움
- **k3d** — k3s 기반, 가장 가벼움
- **Docker Desktop K8s** — 가장 간단

💡 로컬 개발은 minikube/kind, CI는 별도 cluster.

17.5.3 — 매니페스트 버전 관리

규칙:

1. **Git**에 모든 매니페스트 (kubectl apply 기준)
2. **secret**만 별도 (sealed-secrets, ExternalSecrets, AWS Secrets Manager)
3. 환경별 분리 (dev/staging/prod 매니페스트)
4. **kubectl apply -k** 또는 Helm으로 환경 차이 관리

```
git/  
  base/  
    deployment.yaml  
    service.yaml  
  overlays/  
    prod/kustomization.yaml  
    staging/kustomization.yaml
```

17.5.4 — 1판 Ksonnet → 현실 도구

1판 17.5.4: Ksonnet (Heptio)

- 2019년 Heptio 인수 → 2020년 deprecated

현실 대체:

- **Helm** — chart로 패키징 + values로 환경 차이 (가장 대중)
- **Kustomize** — patch로 환경 차이 (kubectl 내장)
- **ArgoCD / Flux** — GitOps (Git 변경 → 클러스터 동기화)
- **CDK8s** — TypeScript/Python으로 매니페스트 생성

💡 17장의 Ksonnet 부분은 "지금은 Helm 또는 Kustomize" 로 멘탈 매핑.

17.5.5 — CI/CD

전형적 흐름:

```
Git push → CI 빌드 (test, image build, push to ECR)
  → CD trigger (ArgoCD/Flux/Spinnaker)
  → kubectl apply → 파드 갱신
  → rollout 완료 모니터링
```

핵심:

- 이미지 태그 = commit SHA (재현성)
- 매니페스트도 Git에서 관리 (재현성)
- rollback은 매니페스트 revert

💡 17장 출간 후 GitOps(ArgoCD/Flux)가 mainstream — 책 범위 밖이지만 현재 표준.

Part 7

atlas-backend 실 사례 분석

"내가 운영 중인 prod 서비스를 17장 체크리스트로"

사례 소개

atlas-backend:

- AWS EKS prod 클러스터 운영
- 87일째 안정 운영
- 컴포넌트: Spring Boot + PostgreSQL + RabbitMQ + Redis
- namespace: `atlas`
- replicas: 1

오늘 점검: 17장의 모범 사례를 얼마나 따르고 있나?

💡 솔직한 사례 — 잘 된 점 + 개선할 점 모두.

atlas-backend 매니페스트 발췌

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: atlas-backend
  namespace: atlas
spec:
  replicas: 1
  strategy: { type: RollingUpdate }
  revisionHistoryLimit: 5
  template:
    metadata:
      labels: { app: atlas-backend }      # 단일 라벨
    spec:
      serviceAccountName: atlas-backend-sa
      terminationGracePeriodSeconds: 30
      containers:
      - name: backend
        image: <account>.dkr.ecr.ap-northeast-2.amazonaws.com/atlas/backend:latest
        imagePullPolicy: Always
        ports: [{ containerPort: 8080 }]
        resources:
          requests: { cpu: 100m, memory: 384Mi }
          limits:   { cpu: 500m, memory: 768Mi }
        startupProbe: { httpGet: {path: /api/actuator/health, port: 8080}, periodSeconds: 10, failureThreshold: 30 }
        livenessProbe: { httpGet: {path: /api/actuator/health, port: 8080}, initialDelaySeconds: 120, periodSeconds: 30 }
        readinessProbe: { httpGet: {path: /api/actuator/health, port: 8080}, initialDelaySeconds: 30, periodSeconds: 10 }
        # ⚠ lifecycle 없음
        env: [...]      # ConfigMap + Secret 참조
```

17장 체크리스트 — atlas-backend 점검 (1/2)

#	항목	1판 절	결과
1	Liveness probe	17.3	✓
2	Readiness probe	17.3.1	✓ initialDelay 30
3	Startup probe (1.16+)	(책 이후 추가)	✓ failureThreshold 30
4	Resources limits/requests	(16장 연결)	✓ 모두 명시
5	terminationGracePeriodSeconds	17.2.5	● 30(기본) 명시
6	preStop hook	17.2.4	● 없음

→ probe는 모범적, but preStop 누락 → Demo 1에서 본 문제 발생 가능.

17장 체크리스트 — atlas-backend 점검 (2/2)

#	항목	1판 절	결과
7	이미지 태그	17.4.2	● <code>:latest</code>
8	imagePullPolicy	17.4.2	● Always (latest 부수효과)
9	Multi-dimensional labels	17.4.3	● <code>app:</code> 1개
10	Annotations	17.4.4	● system만
11	ConfigMap + Secret 분리	(7장 연결)	✓
12	ServiceAccount 분리	(12장 연결)	✓ <code>atlas-backend-sa</code>
13	RollingUpdate + history	(9장 연결)	✓ revisionHistoryLimit 5

최종: 8/13 ✓, 1/13 ●, 4/13 ●

개선 권장: preStop + 명시 태그

```
spec:
  template:
    spec:
      containers:
      - name: backend
        # ✅ 명시적 태그 (CI 빌드 SHA 또는 시맨틱 버전)
        image: <account>.../atlas/backend:v1.2.3-a1b2c3d
        imagePullPolicy: IfNotPresent      # ✅ latest 아닐 때 권장

        # ✅ preStop hook 추가
        lifecycle:
          preStop:
            exec:
              command: ["sh", "-c", "sleep 5"]

        # ✅ 명시
        terminationGracePeriodSeconds: 30
```

효과:

- 롤백 가능 (어떤 버전인지 명확)
- 무중단 rollout (Demo 1에서 본 그것)
- 보안 (재현성)

개선 권장: 다차원 라벨 + 빌드 정보 annotation

```
metadata:
  labels:
    # ✅ 표준 권장 라벨
    app.kubernetes.io/name: atlas-backend
    app.kubernetes.io/instance: atlas-prod
    app.kubernetes.io/version: "1.2.3"
    app.kubernetes.io/component: backend
    app.kubernetes.io/part-of: atlas
    app.kubernetes.io/managed-by: kubectl
  annotations:
    # ✅ 빌드 정보 - 사고 시 즉시 추적 가능
    build.git-commit: "a1b2c3d"
    build.timestamp: "2026-06-13T10:00:00Z"
    contact: "team-platform@example.com"
```

효과:

- 모니터링 dashboard가 자동 그룹핑
- Helm/ArgoCD 호환
- 사고 시 commit 즉시 확인

청중에게 — 점검 명령어

본인 운영 워크로드 한 번 점검해보세요:

```
# 1. 17장 체크리스트 빠르게 확인
kubectl get deploy <name> -n <ns> -o jsonpath='
PRE-STOP: {.spec.template.spec.containers[0].lifecycle.preStop}
IMAGE: {.spec.template.spec.containers[0].image}
LABELS: {.spec.template.metadata.labels}
ANNOTATIONS: {.spec.template.metadata.annotations}
'

# 2. 응답 패턴 (good vs bad)
# - PRE-STOP: 비어있으면 추가 권장
# - IMAGE: latest면 명시적 태그로
# - LABELS: app: 1개면 다차원으로
# - ANNOTATIONS: 비어있으면 빌드 정보 추가
```

→ 5분 점검 → 운영 사고 1회 예방.

Part 8

종합

17장 흔히 놓치는 디테일 모음

#	디테일	#	디테일
1	"앱은 언제나 죽는다" 가정 — 가장 중요	8	표준 라벨 <code>app.kubernetes.io/*</code> 6종
2	endpoint 전파 지연 → preStop sleep 5초 필요	9	빌드 정보는 annotation으로
3	<code>preStop</code> 은 grace period 안에서 카운트	10	로그는 stdout/stderr — 파일 X
4	liveness/readinessProbe endpoint 분리 권장	11	exitCode 137=OOMKilled, 143=SIGTERM 정상
5	startupProbe로 느린 시작 보호	12	1판 Ksonnet → Helm/Kustomize/ArgoCD
6	<code>:latest</code> 절대 금지 (17.4.2)	13	시작 순서는 K8s 보장 X — init container
7	imagePullPolicy: latest면 자동 Always		

→ 인쇄해서 책상에 붙이기.

1판 vs 현실 차이 정리

1판 내용	현실 (2026 기준)
Ksonnet (17.5.4)	 deprecated → Helm/Kustomize/ArgoCD
Liveness + Readiness (2종)	+ Startup probe (1.16 GA) 추가
단일 라벨 권장 안 강조	<code>app.kubernetes.io/*</code> 표준 정립
imagePullPolicy IfNotPresent	그대로 — 단 명시 태그 사용 시
Pod lifecycle	(1판 시점 없던) ephemeral container, sidecar 정식 지원 (1.28)
Logs는 stdout (12-factor)	그대로 — Loki/Vector/Fluent Bit 같은 수집기 발전

💡 책 보면서 Ksonnet 부분은 "지금은 Helm/Kustomize" 멘탈 매핑.

다음 단계 (참고)

이번 발표 범위 밖:

- **GitOps (ArgoCD, Flux)** — 매니페스트 → 클러스터 자동 동기화
- **Pod Disruption Budget** — 정비 시 동시 종료 제한
- **HorizontalPodAutoscaler / VPA** — 자동 스케일
- **Observability** — Prometheus + Grafana + Loki + Tempo
- **Service Mesh (Istio, Linkerd)** — mTLS, 트래픽 정책
- **Cert-manager** — TLS 자동 발급
- **External Secrets Operator** — AWS Secrets Manager 연동

Q&A

🙏 감사합니다